

We'll also define a variable called `insert_location`. The goal of our algorithm is to determine the proper value of `insert_location`, and then it's a simple matter of inserting the file at the `insert_location`. We'll start out by assuming the `insert_location` is 0:

```
insert_location = 0
```

Then we can use a simple `if` statement to check whether the file to insert is higher than the file at the `check_location`. As soon as we encounter a number that's lower than the number to insert, we use its location to decide where to insert our new number. We add 1 because our insertion takes place just behind the lower number we found:

```
if to_insert > cabinet[check_location]:  
    insert_location = check_location + 1
```

After we know the right `insert_location`, we can use a built-in Python method for list manipulation called `insert` to put the file into the cabinet:

```
cabinet.insert(insert_location,to_insert)
```

Running this code will not work to insert our file properly yet, however. We need to put these steps together in one coherent insertion function. This function combines all of the previous code and also adds a `while` loop. The `while` loop is used to iterate over the files in the cabinet, starting with the last file and proceeding until either we find the right `insert_location` or we have examined every file. The final code for our cabinet insertion is in [Listing 4-1](#).

```
def insert_cabinet(cabinet,to_insert):  
    check_location = len(cabinet) - 1  
    insert_location = 0  
    while(check_location >= 0):  
        if to_insert > cabinet[check_location]:  
            insert_location = check_location + 1  
            check_location = - 1  
        check_location = check_location - 1  
    cabinet.insert(insert_location,to_insert)  
    return(cabinet)
```

```
cabinet = [1,2,3,3,4,6,8,12]
```